Slotbook: Product Requirements Document

Version 1.0 · Nov 6, 2026 · Owner: @Rachit · Status: Approved for build

1. Overview

Slotbook is a small Calendly-style scheduler. Hosts publish when they are free, guests book a slot, and nobody is ever double-booked, even when the network, the clock, or the user misbehaves. The booking engine is the product: it must be trustworthy enough to handle money later. The UI is intentionally plain.

One-line pitch: a scheduler whose correctness you can prove with tests.

2. Goals and non-goals

Goals

- G1. Zero double-bookings and zero capacity overruns under concurrency, retries, expiry races and crashes.
- G2. Correct time zone and DST behavior for hosts and guests.
- G3. A clean full-stack TypeScript app on Postgres 15+, runnable with one command.
- G4. Every reliability claim is backed by an automated test against real Postgres.
- G5. Complete submission: repo, README, AI_LOG, report (6-12 pages), full commit history.

Non-goals (core)

Calendar sync, payments, real email delivery, multi-host round-robin, Redis or any external lock service, polished visual design.

3. Users

Persona	Description	Key need
Host	Authenticated person who publishes availability	Set it once, never get double-booked, see and cancel bookings
Guest	Anonymous visitor with a booking link	See slots in my own time zone, book fast, manage later without an account
Reviewer	Evaluates the submission	Run it in one command, read the proof of reliability

4. Product surfaces

1. **Host dashboard** (authenticated): profile and time zone, event types, availability, overrides, bookings.
2. **Public booking page** at `/{host}/{event-slug}`: slot picker, time zone switcher, hold countdown, details form.
3. **Manage page** at `/manage/{token}`: view, reschedule, cancel.
4. **JSON API** used by all three.

5. Functional requirements

5.1 Host

ID	Requirement
H1	Log in with a seeded account (email and password, session cookie).
H2	Set a home time zone as an IANA name (e.g. Asia/Kolkata); invalid names are rejected.

ID	Requirement
H3	Create, edit and archive event types with: name, slug (unique per host), duration 15-120 min, capacity >= 1 (1 = one-on-one, more = group), buffer before and after 0-30 min, minimum notice (e.g. 4 h), booking horizon (e.g. 30 days).
H4	Define weekly availability in host local time, with several windows per weekday (e.g. Mon-Fri 09:00-17:00, or 09:00-12:00 plus 14:00-17:00).
H5	Define date overrides: a full day off, or custom hours on one specific date. An override replaces that date's weekly windows.
H6	View upcoming bookings (guest, event, time in host zone, status).
H7	Cancel a booking with a required reason; the guest is notified.

5.2 Guest

ID	Requirement
G-1	Open /{host}/{slug} and see available slots for the next 30 days (or the event's horizon if shorter), grouped by day in the guest's browser time zone.
G-2	Switch time zone manually; only display changes, never the underlying slot.
G-3	Picking a slot places a 5-minute hold with a visible countdown.
G-4	Enter name and email, then confirm. Confirmation returns a private manage link (shown once).
G-5	If the hold expires, the UI says so and offers fresh slots.
G-6	Via the manage link: view booking, reschedule to a new slot, cancel.
G-7	Group sessions show remaining seats; full slots are not offered.

5.3 Booking lifecycle

```
held --confirm--> confirmed --cancel--> cancelled
  |
  +--timeout--> expired
```

- Capacity counts **held** (not yet expired) plus **confirmed**. It never counts expired or cancelled.
- Expiry is evaluated on the database clock, never the client's.
- Each booking occupies one seat.
- Reschedule moves a confirmed booking atomically; it never leaves the guest with zero or two slots.

5.4 Notifications

- Emit an event on confirm, reschedule and cancel.
- Sink: a notifications table (outbox) plus delivery to a local webhook receiver.
- Events are written in the same transaction as the state change and delivered at least once with retries; each carries a unique ID for dedupe.

6. API

All responses are JSON. Times are UTC ISO-8601 instants; time zones only affect grouping and display.

Method	Path	Purpose	Notes
GET	/api/hosts/{host}/events/{slug}/slots?from&to&tz	Available slots	from/to clamped to horizon; invalid tz returns 400; includes seats left
POST	/api/holds	Hold a slot	Requires Idempotency-Key header; body: host, event slug, slot start

Method	Path	Purpose	Notes
POST	/api/holds/{id}/confirm	Confirm with guest details	Returns manage token once; 410 if expired
GET	/api/bookings/{token}	View booking	Token is hashed in DB
POST	/api/bookings/{token}/reschedule	Move to new slot	Atomic
POST	/api/bookings/{token}/cancel	Cancel	Idempotent
POST	/api/auth/login, /logout	Host session	
GET/PUT/POST	/api/host/...	Profile, event types, availability, overrides, bookings, cancel	Auth required
GET	/health	Liveness and DB check	

Error contract: 400 validation or tampered slot, 401/403 auth, 404 unknown, 409 slot full or conflict, 410 hold expired, 422 idempotency key reused with a different body, 429 rate limited.

7. Reliability requirements

ID	Requirement	Proven by
R1	50 concurrent holds on one capacity-1 slot: exactly 1 succeeds.	concurrency test
R2	Capacity 3 with 20 racers: exactly 3 succeed.	concurrency test
R3	Overlapping slots from different event types of one host (including buffers) never both book.	cross-type race test
R4	10 parallel hold requests with the same Idempotency-Key produce one hold and identical responses.	idempotency test
R5	Same key with a different body returns 422.	idempotency test
R6	Confirm after expiry returns 410 and the slot becomes bookable again.	expiry test
R7	Confirm racing the expiry boundary resolves to exactly one outcome.	boundary test
R8	Reschedule into a taken slot fails and leaves the original booking intact.	reschedule test
R9	Concurrent reschedules into one free slot: one winner.	reschedule race test
R10	A raw SQL insert that overlaps an active session is rejected by the database.	constraint test
R11	A crash or error mid-operation leaves no partial state.	rollback test
R12	If the notification sink is down, events are delivered after recovery with no duplicates.	outbox test
R13	A tampered start time, off-grid time, past time, or time outside notice and horizon is rejected.	validation test
R14	Correctness does not depend on the sweeper or any cron job running.	sweeper-disabled test

Design approach

1. **Per-host lock:** every write transaction starts with `SELECT ... FROM hosts WHERE id = $1 FOR UPDATE`, serializing writes per host.
2. **Capacity check inside the lock** using DB now().
3. **Database backstop:** a GiST exclusion constraint on (host_id, blocked_range) for active sessions, where blocked_range is the slot plus buffers.
4. **Idempotency** record inserted first in the same transaction as the hold.
5. **Transactional outbox** for notifications, delivered with FOR UPDATE SKIP LOCKED.
6. **Lazy sessions:** a session row is created on the first hold and shared by group bookings.

8. Time zone requirements

ID	Requirement
T1	Availability is stored as local wall-clock times plus the host's IANA zone, never as fixed UTC.
T2	Slots are generated per date in the host zone, converted to UTC instants, and stepped in absolute time.
T3	Slot step equals event duration.
T4	Spring-forward: nonexistent local times produce no slot. Fall-back: the repeated hour is handled without duplicate or missing slots.
T5	Guest tz affects display and day grouping only.

Worked example (for the report): a New York host with a 09:00 slot. On Sat Mar 7, 2026 (EST, UTC-5) it is 14:00Z, shown as 19:30 to an Asia/Kolkata guest. On Mon Mar 9, 2026 (EDT, UTC-4) it is 13:00Z, shown as 18:30. The host's wall clock stays 09:00 while the UTC instant shifts.

9. Data model

Table	Key columns and constraints
hosts	slug (unique), email (unique), password_hash, timezone
event_types	host_id, slug (unique per host), duration 15-120, capacity >= 1, buffer_before/after 0-30, min_notice, horizon_days, archived_at
weekly_availability	host_id, weekday, start_time, end_time; check start < end; no overlapping windows per day
date_overrides	host_id, date, is_off, optional windows
sessions	host_id, event_type_id, start_at, end_at, blocked_range, capacity, status; **exclusion constraint** on (host_id, blocked_range) where active; unique (event_type_id, start_at) where active
bookings	session_id, status, hold_expires_at, guest_name, guest_email, token_hash, cancel_reason; check constraints tie status to required fields
idempotency_keys	key (unique), request_hash, response, created_at
notifications	id, type, payload, status, attempts, next_attempt_at, delivered_at

10. Non-functional requirements

- **Run:** docker compose up starts app and Postgres, runs migrations, seeds a demo host with two event types.
- **Stack:** TypeScript end to end; Fastify API, React (Vite) frontend, Postgres 15+, plain SQL migrations, zod validation, Luxon for time.
- **Security:** hashed passwords, hashed manage tokens, parameterized queries, input validation on every route.
- **Performance target:** slot listing p95 under 300 ms; hold p95 under 200 ms at moderate concurrency (measured with k6).
- **Observability:** structured logs with request IDs, /health.
- **Tests:** one command runs everything, including concurrency tests against real Postgres.

11. Optional extensions (planned)

Extension	Why	Risk to core
Observability (logs, /health, basic metrics)	Cheap, high signal	None
Daily cap and per-guest limit	A few checks under the host lock	Low
Abuse protection (rate limits, per-IP hold cap)	Stops bots hoarding slots	Low

Extension	Why	Risk to core
Load test (k6) with numbers	Evidence for reliability claims	None
End-to-end tests and GitHub Actions CI	Proves the guest flow	None

Deliberately skipped: waitlist, host webhooks, round-robin, reminders, external busy times. They add risk to the core within the timebox.

12. Acceptance criteria

- All requirements R1-R14 pass in automated tests.
- Fresh clone, docker compose up, then the demo booking flow works end to end with no manual steps.
- DST spring and fall tests pass.
- README documents run steps, API and a 60-second UI tour.
- AI_LOG.md lists key prompts, where AI was wrong, and what was changed by hand.
- Report contains all ten required sections.

13. Assumptions

- Slot step equals event duration.
- Each booking takes one seat.
- Guest emails are not verified.
- Editing availability or an event type never alters existing bookings.
- Hold TTL is fixed at 5 minutes.
- Buffer conflict detection is deliberately conservative.
- One host login per seeded account; no self-signup.

14. Risks and mitigations

Risk	Mitigation
DST edge cases produce wrong slots	Pure slot function with table-driven unit tests for both transitions
Lock contention slows a very busy host	Lock is per host and held only for short transactions; measure in load test
Scope creep eats the 46 h window	Core first; extensions only after R1-R14 are green
Reliance on AI introduces subtle bugs	Constraint backstop in the DB; tests written from the failure table, reviewed by hand

15. Milestones (about 6 hours of build)

Phase	Time	Output
0. Scaffold	30 min	Repo, compose, migration runner, seed, Fastify and Vite skeleton
1. Schema and slot engine	60 min	Migrations, generateSlots, DST, override, notice and horizon tests
2. Booking engine	90 min	Hold, confirm, cancel, reschedule, idempotency, outbox, concurrency tests
3. API and auth	45 min	zod validation, login, host and public routes
4. Frontend	75 min	Dashboard, booking page with countdown and tz switcher, manage page
5. Workers and extras	30 min	Sweeper, outbox worker, webhook receiver, /health, request-ID logs
6. Docs	45 min	README, API doc, AI_LOG, report PDF with diagram

Remaining time in the 46 h window is buffer for the optional extensions, load test and report polish.

16. Deliverables checklist

- [ ] Public GitHub repo with full, unsquashed history
- [ ] Source, migrations, seed script
- [ ] One-command tests including concurrency against real Postgres
- [ ] README.md
- [ ] AI_LOG.md
- [ ] Report PDF (6-12 pages) linking the repo
- [ ] List of optional extensions built
- [ ] Report and repo sent to the recruiter